Index

A

abstractness (modularity metric), Core Metrics
accidental complexity, Distance from the Main Sequence, The 4 Cs of Architecture
Accidental SOA antipattern, Common Risks
ACID (atomicity, consistency, isolation, durability) database transactions, Style Specifics, Style Characteristics
active/active state, Data Collisions
ADRs (see Architectural Decision Records)
Advanced Message Queueing Protocol (AMQP), Preventing Data Loss
afferent coupling, Coupling, Static Coupling
Agile methodologies, Architecture and Engineering Practices
agility, Architecture and Engineering Practices
Ambulance pattern, Elasticity
AMQP (Advanced Message Queueing Protocol), Preventing Data Loss
Analysis Paralysis antipattern, The Covering Your Assets Antipattern
anemic events, Anemic events
antipatterns
    Accidental SOA, Common Risks
    Analysis Paralysis, The Covering Your Assets Antipattern
    architectural decision-related, Architectural Decision Antipatterns-Email-Driven Architecture Antipattern
    Architecture Sinkhole, Adding Layers
    Big Ball of Mud, Big Ball of Mud-Big Ball of Mud
    Bottleneck Trap, Balancing Architecture and Hands-On Coding
    Covering Your Assets, The Covering Your Assets Antipattern-The Covering Your Assets Antipattern
    defined, Architectural Decision Antipatterns
    DLL Hell, Peer-to-peer approach
    Email-Driven Architecture, Email-Driven Architecture Antipattern
    Entity Trap, The Entity Trap-The Entity Trap, Data Isolation

Frozen Caveman, Technical Breadth
generic architecture, Limiting and Prioritizing Architectural Characteristics
Grains of Sand, Common Risks
Groundhog Day, Groundhog Day Antipattern
Irrational Artifact Attachment, Tools
Out of Context, Out of Context Antipattern
Swarm of Gnats, The Swarm of Gnats Antipattern-The Swarm of Gnats Antipattern
Apache Kafka, Examples and Use Cases-Examples and Use Cases
application services, Application services
application silos, Understand and Navigate Politics
ArchiMate technical diagramming standard, ArchiMate
architects (see software architect)
architectural characteristics
analyzing when creating logical components, Analyzing Architectural Characteristics
defined, Architectural Characteristics Defined-Trade-Offs and Least Worst Architecture
cloud characteristics, Cloud Characteristics
cross-cutting architectural characteristics, Cross-Cutting Architectural Characteristics-Cross-Cutting Architectural Characteristics
non-functional requirements, as term, Architectural Characteristics Defined
operational architectural characteristics, Operational Architectural Characteristics
partial listing of characteristics, Architectural Characteristics (Partially) Listed-Cross-Cutting Architectural Characteristics
self-assessment questions, Chapter 4: Architecture Characteristics Defined
structural architectural characteristics, Structural Architectural Characteristics
system design, Architectural Characteristics and System Design-Architectural Characteristics and System Design
defining, Defining Software Architecture
fitness functions, Fitness Functions-Distance from the Main Sequence fitness function

identifying, Identifying Architectural Characteristics-Limiting and Prioritizing Architectural Characteristics
architecture katas origins, Extracting Architectural Characteristics
composite architectural characteristics, Composite Architectural Characteristics-Composite Architectural Characteristics
design versus architecture and trade-offs, Implicit Characteristics
extracting architectural characteristics, Extracting Architectural Characteristics
extracting architectural characteristics from domain concerns, Extracting Architectural Characteristics from Domain Concerns
kata: Silicon Sandwiches, Kata: Silicon Sandwiches-Implicit Characteristics
limiting/prioritizing architectural characteristics, Limiting and Prioritizing Architectural Characteristics-Limiting and Prioritizing Architectural Characteristics
self-assessment questions, Chapter 5: Identifying Architectural Characteristics
working with katas, Working with Katas
intersection of architecture with data topologies, Architectural Characteristics
ISO definitions, Cross-Cutting Architectural Characteristics-Cross-Cutting Architectural Characteristics
measuring, Measuring and Governing Architecture Characteristics-Process Measures
cyclomatic complexity, Structural Measures-Structural Measures
multiple definitions of performance, Operational Measures
operational measures, Operational Measures-Operational Measures
process measures, Process Measures
self-assessment questions, Chapter 6: Measuring and Governing Architecture Characteristics
structural measures, Structural Measures-Structural Measures
overspecificity in Vasa case, Limiting and Prioritizing Architectural Characteristics
scope of, The Scope of Architectural Characteristics-Scoping and the Cloud

architectural quanta and granularity, Architectural Quanta and Granularity-Architectural Quanta and Granularity
impact of scoping, The Impact of Scoping-Kata: Going Green
kata: Going Green, Kata: Going Green
scoping and architectural style, Scoping and Architectural Style-Scoping and Architectural Style
scoping and the cloud, Scoping and the Cloud
self-assessment questions, Chapter 7: The Scope of Architectural Characteristics
synchronous communication, Synchronous Communication
trade-offs and least worst architecture, Trade-Offs and Least Worst Architecture-Trade-Offs and Least Worst Architecture
worksheet for determining characteristics critical to success, Limiting and Prioritizing Architectural Characteristics-Limiting and Prioritizing Architectural Characteristics
architectural constraints, Architectural Constraints
Architectural Decision Records (ADRs), Architectural Decision Records-Using ADRs with Existing Systems
basic structure, Basic Structure-Notes
Compliance section, Compliance-Compliance
Consequences section, Consequences
Context section, Context
Decision section, Decision
Notes section, Notes
status section, Status-Status
title section, Title
documentation provided by, ADRs as Documentation
example: Going, Going, Gone, Example-Example
storing, Storing ADRs-Storing ADRs
using for standards, Using ADRs for Standards
using with existing systems, Using ADRs with Existing Systems
architectural decisions, Architectural Decisions-Leveraging Generative AI and LLMs in Architectural Decisions
antipatterns, Architectural Decision Antipatterns-Email-Driven Architecture Antipattern
Covering Your Assets, The Covering Your Assets Antipattern-The Covering Your Assets Antipattern
Email-Driven Architecture antipattern, Email-Driven Architecture Antipattern

Groundhog Day antipattern, Groundhog Day Antipattern
Architectural Decision Records, Architectural Decision Records-Using ADRs with Existing Systems
as documentation, Storing ADRs-Storing ADRs
basic structure, Basic Structure-Notes
example: Going, Going, Gone, Example-Example
storing ADRs, Storing ADRs-Storing ADRs
using ADRs for standards, Using ADRs for Standards
using with existing systems, Using ADRs with Existing Systems
architecturally significant decisions, Architectural Significance-Architectural Significance
defining, Defining Software Architecture
leveraging generative AI and LLMs in, Leveraging Generative AI and LLMs in Architectural Decisions
self-assessment questions, Chapter 21: Architectural Decisions
architectural extensibility, Analyzing Trade-Offs, Topology
architectural fitness function, Fitness Functions, Architecture and Engineering Practices
(see also fitness functions)
architectural intersections (see intersections of architecture)
architectural katas (see katas)
architectural patterns, Architectural Patterns-Broker-Domain Pattern
architectural styles versus, Styles Versus Patterns-Styles Versus Patterns, Architectural Patterns
communication, Communication-Orchestration Versus Choreography
CQRS, CQRS
infrastructure, Infrastructure-Broker-Domain Pattern
Broker–Domain pattern, Broker-Domain Pattern-Broker-Domain Pattern
Domain–Broker pattern, Broker-Domain Pattern-Broker-Domain Pattern
orchestration versus choreography, Orchestration Versus Choreography-Orchestration Versus Choreography
reuse, Reuse-Service Mesh
separating domain and operational coupling, Separating Domain and Operational Coupling-Service Mesh
architectural quanta, Architectural Quanta and Granularity-Architectural Quanta and Granularity

domain-driven design's bounded context, Architectural Quanta and Granularity
scope and, The Impact of Scoping-Kata: Going Green
architectural risk, analyzing, Analyzing Architecture Risk-Summary
risk assessments, Risk Assessments-Risk Assessments
risk matrix, Risk Matrix-Risk Matrix
risk storming, Risk Storming-Phase 3: Risk Mitigation
phase 1: identification, Phase 1: Identification
phase 2: consensus, Phase 2: Consensus-Phase 2: Consensus
phase 3: mitigation, Phase 3: Risk Mitigation
use case, Risk-Storming Use Case-Security
user-story risk analysis, User-Story Risk Analysis
self-assessment questions, Chapter 22: Analyzing Architecture Risk
architectural styles, Defining Software Architecture, Foundations-On to Specific Styles
(see also specific styles, e.g.: event-driven architecture style)
architecture partitioning, Architecture Partitioning-Technical partitioning
architecture patterns and (see architectural patterns)
characteristics described by, Styles Versus Patterns-Styles Versus Patterns
choosing the appropriate style, Choosing the Appropriate Architecture Style-Distributed Case Study: Going, Going, Gone
decision criteria, Decision Criteria-Decision Criteria
distributed case study: Going, Going, Gone, Distributed Case Study: Going, Going, Gone-Distributed Case Study: Going, Going, Gone
factors in shifting fashions in architecture, Shifting “Fashion” in Architecture-Shifting “Fashion” in Architecture
monolith case study: Silicon Sandwiches, Monolith Case Study: Silicon Sandwiches-Microkernel
self-assessment questions, Chapter 19: Choosing the Appropriate Architecture Style
defining, Defining Software Architecture
fundamental patterns, Fundamental Patterns-Three-tier
kata: Silicon Sandwiches, Kata: Silicon Sandwiches—Partitioning-Technical partitioning
self-assessment questions, Chapter 9: Foundations

architectural thinking, Architectural Thinking-There's More to Architectural Thinking
analyzing trade-offs, Analyzing Trade-Offs-Analyzing Trade-Offs
architecture versus design, Architecture Versus Design-The Significance of Trade-Offs
level of effort, Level of Effort
significance of trade-offs, The Significance of Trade-Offs
strategic versus tactical decisions, Strategic Versus Tactical Decisions
balancing architecture and hands-on coding, Balancing Architecture and Hands-On Coding-Balancing Architecture and Hands-On Coding
self-assessment questions, Chapter 2: Architectural Thinking
technical breadth, Technical Breadth-Rings
20-minute rule, The 20-Minute Rule-The 20-Minute Rule
developing a personal radar for technological change, Developing a Personal Radar-Rings
understanding business drivers, Understanding Business Drivers
architecturally significant decisions, Architectural Significance-Architectural Significance
architecture isomorphism, Decision Criteria
architecture risk-assessment matrix, Risk Matrix-Risk Matrix
Architecture Sinkhole antipattern, Adding Layers
architecture vitality, Continually Analyze the Architecture
architecture, design versus, Architecture Versus Design-The Significance of Trade-Offs, Implicit Characteristics
level of effort, Level of Effort
significance of trade-offs, The Significance of Trade-Offs
strategic versus tactical decisions, Strategic Versus Tactical Decisions
ArchUnit, Distance from the Main Sequence fitness function
asynchronous communication, Decision Criteria
auto acknowledge mode, Preventing Data Loss
availability, as implicit characteristic, Implicit Characteristics
AWS Step Functions, Cloud Considerations

B

Backends for Frontends (BFF) pattern, Microkernel
best practices, architecture patterns versus, Architectural Patterns

Big Ball of Mud antipattern, Big Ball of Mud-Big Ball of Mud
Bottleneck Trap antipattern, Balancing Architecture and Hands-On Coding
bounded context, Bounded Context
domain-driven design and, Architectural Quanta and Granularity
microservices and, Microservices Architecture
BPEL (Business Process Execution Language), Mediated Event-Driven Architecture
BPM (Business Process Management), Mediated Event-Driven Architecture
brittle architecture, Architectural Quanta and Granularity
business drivers, as part of architectural thinking, Understanding Business Drivers
business environment, intersection of architecture with, Architecture and the Business Environment-Architecture and the Business Environment
Business Process Execution Language (BPEL), Mediated Event-Driven Architecture
Business Process Management (BPM), Mediated Event-Driven Architecture
business services, Business services
business stakeholders, negotiating with, Negotiating with Business Stakeholders-Negotiating with Business Stakeholders
business-domain expertise, Know the Business Domain

C

C4 (context, container, component, class) technical diagramming standard, C4
caching
near-cache considerations, Near-cache considerations
replicated and distributed caching, Replicated and distributed caching-Replicated and distributed caching
CC (see Cyclomatic Complexity)
chaos engineering, Distance from the Main Sequence fitness function
checklists
fitness functions as, Distance from the Main Sequence fitness function
leveraging, Leveraging Checklists-Software-Release Checklist

developer code-completion checklist, Developer Code-Completion Checklist-Developer Code-Completion Checklist
Hawthorne effect, Leveraging Checklists
software release checklist, Software-Release Checklist
unit and functional testing checklist, Unit and Functional Testing Checklist
Chidamber and Kemerer Object-Oriented Metrics Suite, Cohesion
choreography
in microservices, Choreography and Orchestration-Choreography and Orchestration
orchestration versus, Orchestration Versus Choreography-Orchestration Versus Choreography
CID (correlation ID), Request-Reply Processing
client acknowledge mode, Preventing Data Loss
client/server architecture, Client/Server-Three-tier
browser and web server, Browser and web server
desktop and database server, Desktop and database server
single-page JavaScript applications, Single-page JavaScript applications
three-tier architecture, Three-tier
closed layers, Layers of Isolation
cloud environments
architectural characteristics, Cloud Characteristics
event-driven architecture, Cloud Considerations
intersection of architecture and infrastructure, Architecture and Infrastructure
layered architecture, Cloud Considerations
microkernel architecture, Cloud Considerations
microservices architecture, Cloud Considerations
modular monolith architecture, Cloud Considerations
orchestration-driven service-oriented architecture, Cloud Considerations
pipeline architecture, Cloud Considerations-Cloud Considerations
scoping and, Scoping and the Cloud
service-based architecture, Cloud Considerations
space-based architecture, Cloud Considerations
code reviews, Balancing Architecture and Hands-On Coding
code smell, Structural Measures

coding, balancing architecture work with, Balancing Architecture and Hands-On Coding-Balancing Architecture and Hands-On Coding
cohesion, defined, Architectural Quanta and Granularity
communication
diagramming architecture (see diagramming architecture)
in microservices, Communication-Communication
synchronous, Synchronous Communication
compatibility, ISO definition, Cross-Cutting Architectural Characteristics
compensating transaction framework, Transactions and Sagas
compensating updates, Fallacy #10. Compensating updates always work
compile-based plug-in components, Plug-In Components
complexity, cyclomatic, Structural Measures-Structural Measures
compliance with design principles/architecture decisions, Ensure Compliance with Decisions
complicated-subsystem teams, defined, Team Topologies and Architecture
component coupling, Component Coupling-The Law of Demeter
Law of Demeter, The Law of Demeter-The Law of Demeter
static coupling, Static Coupling
temporal coupling, Temporal Coupling
component identification, Identifying Core Components-The Entity Trap
Actor/Action approach, The Actor/Action approach-The Actor/Action approach
Entity Trap antipattern, The Entity Trap-The Entity Trap
Workflow approach, The Workflow approach
component-based thinking, Component-Based Thinking-Case Study: Going, Going, Gone—Discovering Components
case study: Going, Going, Gone (discovering components), Case Study: Going, Going, Gone—Discovering Components-Case Study: Going, Going, Gone—Discovering Components
component coupling, Component Coupling-The Law of Demeter
Law of Demeter, The Law of Demeter-The Law of Demeter
static coupling, Static Coupling
temporal coupling, Temporal Coupling
creating a logical architecture, Creating a Logical Architecture-Restructuring Components

analyzing architectural characteristics, Analyzing Architectural Characteristics
analyzing roles/responsibilities, Analyzing Roles and Responsibilities-Analyzing Roles and Responsibilities
assigning user stories to components, Assigning User Stories to Components-Assigning User Stories to Components
identifying core components, Identifying Core Components-The Entity Trap
restructuring components, Restructuring Components
defining logical components, Defining Logical Components-Defining Logical Components
logical versus physical architecture, Logical Versus Physical Architecture-Logical Versus Physical Architecture
self-assessment questions, Chapter 8: Component-Based Thinking
components, modules and, From Modules to Components
composite architectural characteristics, Composite Architectural Characteristics-Composite Architectural Characteristics
concert ticketing system use case, Concert Ticketing System
connascence, Connascence-Connascence properties
dynamic, Dynamic connascence-Dynamic connascence
properties, Connascence properties-Connascence properties
static, Static connascence-Static connascence
connascence of values, Transactions and Sagas
constraints, defined, Architectural Constraints
construction techniques, defined, Architectural Significance
consumer filters, Filters
Conway's Law, Architecture Partitioning
correlation ID (CID), Request-Reply Processing
coupling, Component Coupling-The Law of Demeter
architectural quanta and, Architectural Quanta and Granularity-Architectural Quanta and Granularity
component coupling, Component Coupling-The Law of Demeter
Law of Demeter, The Law of Demeter-The Law of Demeter
static coupling, Static Coupling
temporal coupling, Temporal Coupling
orthogonal, Service Mesh
separating domain coupling and operational coupling, Separating Domain and Operational Coupling-Service Mesh

Hexagonal architecture, Hexagonal architecture-Hexagonal architecture
Service Mesh, Service Mesh-Service Mesh
static, Static Coupling
temporal, Temporal Coupling
Covering Your Assets antipattern, The Covering Your Assets Antipattern-The Covering Your Assets Antipattern
CQRS (Command–Query–Responsibility–Segregation) pattern, CQRS
cross-cutting architectural characteristics, Cross-Cutting Architectural Characteristics-Cross-Cutting Architectural Characteristics
customizability, Implicit Characteristics
Cyclic Dependencies antipattern, Cyclic dependencies-Cyclic dependencies
Cyclomatic Complexity (CC), Structural Measures-Structural Measures
defined, Structural Measures
formula for calculating, Structural Measures
limitations of metric, Distance from the Main Sequence
threshold value for, Structural Measures

D

data abstraction layer, Data Readers
data collisions, Data Collisions-Data Collisions
data isolation, Data Isolation
data latency, Style Characteristics
data pumps, Data Pumps-Data Pumps
data topologies, intersection of architecture with, Architecture and Data Topologies-Read/Write Priority
architectural characteristics, Architectural Characteristics
data structure, Data Structure
database topologies, Database Topology-Database Topology
read/write priority, Read/Write Priority
data-based event payload, Data-based event payloads-Data-based event payloads
Database-per-Service pattern, Dedicated Data Topology-Dedicated Data Topology, Data Topologies-Data Topologies
DDD (see domain-driven design)
decisions, strategic versus tactical, Strategic Versus Tactical Decisions
declarative transactions, Data Topologies

dedicated database topology, Dedicated Data Topology-Dedicated Data Topology
dependencies, defined, Architectural Significance
dependency-free plug-ins, Plug-In Dependencies
derived event, Topology
design principles
architect's role in defining, Make Architecture Decisions
architect's role in ensuring compliance with, Ensure Compliance with Decisions
communicating to team, Providing Guidance-Providing Guidance
design, architecture versus, Architecture Versus Design-The Significance of Trade-Offs, Implicit Characteristics
developer code-completion checklist, Developer Code-Completion Checklist-Developer Code-Completion Checklist
developers
flow state, Integrating with the Development Team
negotiating with, Negotiating with Developers-Negotiating with Developers
technical depth versus technical breadth, Technical Breadth
development teams, making effective, Making Teams Effective-Summary
appropriate level of involvement for software engineer, How Much Involvement?-How Much Involvement?
architect personality types, Architect Personalities-The Effective Architect
armchair architect, The Armchair Architect-The Armchair Architect
control-freak architect, The Control-Freak Architect
effective architect, The Effective Architect
collaboration, Collaboration-Collaboration
constraints and boundaries, Constraints and Boundaries-Constraints and Boundaries
impact of business justifications, Providing Guidance
leading by example, Leading Teams by Example-Leading Teams by Example
leveraging checklists, Leveraging Checklists-Software-Release Checklist
developer code-completion checklist, Developer Code-Completion Checklist-Developer Code-Completion Checklist

Hawthorne effect, Leveraging Checklists
software release checklist, Software-Release Checklist
unit and functional testing checklist, Unit and Functional Testing Checklist
negotiating with teams, Negotiating with Developers-Negotiating with Developers
providing guidance, Providing Guidance-Providing Guidance
self-assessment questions, Chapter 24: Making Teams Effective
team topologies (see team topologies)
warning signs of too-large team, Team Warning Signs-Pluralistic Ignorance
diffusion of responsibility, Pluralistic Ignorance
pluralistic ignorance, Pluralistic Ignorance
process loss, Process Loss
DevOps, Architecture and Infrastructure
diagramming architecture, Diagramming Architecture-Summary
guidelines, Diagram Guidelines-Keys
self-assessment questions, Chapter 23: Diagramming Architecture
standards, Diagramming Standards: UML, C4, and ArchiMate-ArchiMate
ArchiMate, ArchiMate
C4, C4
UML, UML
tools, Tools-Tools
using layers semantically, Tools
diffusion of responsibility, Pluralistic Ignorance
direction of risk, Risk Assessments-Risk Assessments
Distance from the Main Sequence metric
basics, Distance from the Main Sequence-Distance from the Main Sequence
verification using fitness functions, Distance from the Main Sequence fitness function-Distance from the Main Sequence fitness function
distributed architectures
Going, Going, Gone case study, Distributed Case Study: Going, Going, Gone-Distributed Case Study: Going, Going, Gone
microservices as, Topology
monolithic architectures versus, Monolithic Versus Distributed Architectures-Fallacy #11. Observability is optional (for distributed

architectures)
fallacy #1: reliable network, Fallacy #1: The Network Is Reliable
fallacy #2: zero latency, Fallacy #2: Latency Is Zero
fallacy #3: infinite bandwidth, Fallacy #3: Bandwidth Is Infinite
fallacy #4: secure network, Fallacy #4: The Network Is Secure
fallacy #5: unchanging topology, Fallacy #5: The Topology Never Changes
fallacy #6: single administrator, Fallacy #6: There Is Only One Administrator
fallacy #7: zero transport cost, Fallacy #7: Transport Cost Is Zero
fallacy #8: homogeneous network, Fallacy #8: The Network Is Homogeneous
fallacy #9: easy versioning, Fallacy #9. Versioning is easy
fallacy #10: reliable compensating updates, Fallacy #10. Compensating updates always work
fallacy #11: observability as optional, Fallacy #11. Observability is optional (for distributed architectures)
scoping and, Scoping and Architectural Style
distributed caching, Replicated and distributed caching-Replicated and distributed caching
DLL Hell antipattern, Peer-to-peer approach
domain coupling, separating operational coupling from, Separating Domain and Operational Coupling-Service Mesh
domain database topology, Domain Database Topology-Domain Database Topology
domain partitioning, Architecture Partitioning-Architecture Partitioning, Domain partitioning
microkernel architecture and, Architecture Characteristics Ratings
modular monolithic topology, Topology
service-based architecture and, Style Characteristics
teams and, Architecture and Team Topologies
domain services, Topology
domain-driven design (DDD)
bounded context and, Architectural Quanta and Granularity
domain partitioning in, Architecture Partitioning
microservices and, Microservices Architecture
modular monolithic architecture and, The Modular Monolith Architecture Style

domain-to-architecture isomorphism, Architecture and the Business Environment
domains, in modular monolithic architecture, Style Specifics
durable subscriber, Preventing Data Loss
dynamic connascence, Dynamic connascence-Dynamic connascence, Transactions and Sagas
dynamic coupling, Architectural Quanta and Granularity
  communication as, Synchronous Communication
  in microservices, Governance
dynamic quantum entanglement, Asynchronous Capabilities

E

EasyMeals use case, Examples and Use Cases-Examples and Use Cases
EDA (see event-driven architecture (EDA) style)
EDI (electronic data interchange) tools, When Not to Use
effective architect personality type, The Effective Architect
efferent coupling, Coupling, Static Coupling
Elastic Leadership, How Much Involvement?
elastic scale, Architecture and Infrastructure
elasticity, Explicit Characteristics-Explicit Characteristics
electronic data interchange (EDI) tools, When Not to Use
Email-Driven Architecture antipattern, Email-Driven Architecture Antipattern
enabling teams, defined, Team Topologies and Architecture
enforced heterogeneity, Communication
engineering practices
  intersection of architecture with, Architecture and Engineering Practices-Architecture and Engineering Practices
  software engineering and, Architecture and Engineering Practices
ensuring compliance, Ensure Compliance with Decisions
enterprise service bus (ESB), Orchestration engine and message bus, Governance-Governance
enterprise services, Enterprise services
enterprise, intersection of architecture with, Architecture and the Enterprise
Entity Trap antipattern, The Entity Trap-The Entity Trap, Data Isolation
essential complexity, Distance from the Main Sequence
ETL (extract, transform, and load) tools, When Not to Use

event mediator, Mediated Event-Driven Architecture
event payloads, Event Payload-Anemic events
anemic events, Anemic events
data-based, Data-based event payloads-Data-based event payloads
key-based, Key-based event payload-Key-based event payload
trade-off summary, Trade-off summary
event-based application, Event-Driven Architecture Style
event-driven architecture (EDA) style, Event-Driven Architecture Style-Examples and Use Cases
choosing between request-based and event-based models, Choosing Between Request-Based and Event-Based Models
cloud considerations, Cloud Considerations
common risks, Common Risks
data topologies, Data Topologies-Dedicated Data Topology
dedicated data topology, Dedicated Data Topology-Dedicated Data Topology
domain database topology, Domain Database Topology-Domain Database Topology
monolithic database topology, Monolithic Database Topology-Monolithic Database Topology
examples and use cases, Examples and Use Cases-Examples and Use Cases
governance, Governance
style characteristics, Style Characteristics-Choosing Between Request-Based and Event-Based Models
style specifics, Style Specifics-Mediated Event-Driven Architecture
asynchronous capabilities, Asynchronous Capabilities-Asynchronous Capabilities
broadcast capabilities, Broadcast Capabilities
derived events, Derived Events-Derived Events
error handling, Error Handling-Error Handling
event payload, Event Payload-Anemic events
events versus messages, Events Versus Messages-Events Versus Messages
mediated event-driven architecture, Mediated Event-Driven Architecture-Mediated Event-Driven Architecture
preventing data loss, Preventing Data Loss-Preventing Data Loss
request-reply processing, Request-Reply Processing-Request-Reply Processing

Swarm of Gnats antipattern, The Swarm of Gnats Antipattern-The Swarm of Gnats Antipattern
triggering extensible events, Triggering Extensible Events
team topology considerations, Team Topology Considerations-Team Topology Considerations
topology, Topology-Topology
eventual consistency, Style Characteristics
explicit architectural characteristics, Explicit Characteristics-Explicit Characteristics
extensible derived event, Triggering Extensible Events
extract, transform, and load (ETL) tools, When Not to Use

F

facilitation, as requirement for software architect, Possess Interpersonal Skills
fallacies of distributed computing
fallacy #1: reliable network, Fallacy #1: The Network Is Reliable
fallacy #2: zero latency, Fallacy #2: Latency Is Zero
fallacy #3: infinite bandwidth, Fallacy #3: Bandwidth Is Infinite
fallacy #4: secure network, Fallacy #4: The Network Is Secure
fallacy #5: unchanging topology, Fallacy #5: The Topology Never Changes
fallacy #6: single administrator, Fallacy #6: There Is Only One Administrator
fallacy #7: zero transport cost, Fallacy #7: Transport Cost Is Zero
fallacy #8: homogeneous network, Fallacy #8: The Network Is Homogeneous
fallacy #9: easy versioning, Fallacy #9. Versioning is easy
fallacy #10: reliable compensating updates, Fallacy #10. Compensating updates always work
fallacy #11: observability as optional, Fallacy #11. Observability is optional (for distributed architectures)
fan-in coupling (see afferent coupling)
fan-out coupling (see efferent coupling)
federated broker, Topology
filters, in pipeline architectural style, Filters
fire-and-forget communication, Topology, Asynchronous Capabilities
First Law of Software Architecture, Laws of Software Architecture, First Law: Everything in Software Architecture Is a Trade-Off-Second

Corollary: You Can't Do It Just Once
architecture partitioning and, Architecture Partitioning
example: shared library versus shared service, Shared Library Versus Shared Service-Shared Library Versus Shared Service
example: synchronous versus asynchronous messaging, Synchronous Versus Asynchronous Messaging-Synchronous Versus Asynchronous Messaging
first corollary: missing trade-offs, First Corollary: Missing Trade-Offs-First Corollary: Missing Trade-Offs
second corollary: trade-off analysis as primary job for architects, Second Corollary: You Can't Do It Just Once
fitness functions
architectural characteristics, Fitness Functions-Distance from the Main Sequence fitness function
cyclic dependencies, Cyclic dependencies-Cyclic dependencies
Distance from the Main Sequence, Distance from the Main Sequence fitness function-Distance from the Main Sequence fitness function
flow state, Integrating with the Development Team
4 Cs of architecture leadership (communication, collaboration, clear, concise), The 4 Cs of Architecture-The 4 Cs of Architecture
Front Controller pattern, Choreography and Orchestration
frontends, Frontends-Frontends
Frozen Caveman antipattern, Technical Breadth
full-time equivalency (FTE) rate, Status
function point analysis, Architectural Characteristics Defined
functional cohesion, Architectural Quanta and Granularity, Architectural Quanta and Granularity
functional suitability, Cross-Cutting Architectural Characteristics

G

generative AI
inability to evaluate trade-offs, Introduction
intersection of architecture with, Architecture and Generative AI-Generative AI as an Architect Assistant
Gen AI as architect assistant, Generative AI as an Architect Assistant-Generative AI as an Architect Assistant
incorporating Gen AI into architecture, Incorporating Generative AI into Architecture

leveraging in architectural decisions, Leveraging Generative AI and LLMs in Architectural Decisions
generic architecture antipattern, Limiting and Prioritizing Architectural Characteristics
Going Green (kata)
architecture quantum as scope for architectural characteristics, Kata: Going Green-Kata: Going Green
microkernel architecture for, Core System-Core System
service-based architecture, Style Characteristics-Examples and Use Cases
Going, Going, Gone (GGG)
ADR example, Example-Example
discovering components, Case Study: Going, Going, Gone—Discovering Components-Case Study: Going, Going, Gone—Discovering Components
distributed architectures, Distributed Case Study: Going, Going, Gone-Distributed Case Study: Going, Going, Gone
EDA style, Examples and Use Cases-Examples and Use Cases
governance, Governance and Fitness Functions
architectural characteristics, Governance and Fitness Functions
EDA style, Governance
layered architectural style, Governance
microkernel architectural style, Governance
microservices architectural style, Governance-Governance
modular monolith architectural style, Governance-Governance
orchestration-driven service-oriented architecture, Governance-Governance
pipeline architectural style, Governance-Governance
service-based architectural style, Governance
space-based architecture style, Governance-Governance
Grains of Sand antipattern, Common Risks
granularity
architectural quanta and, Architectural Quanta and Granularity-Architectural Quanta and Granularity
modularity versus, Modularity Versus Granularity
Groundhog Day antipattern, Groundhog Day Antipattern
guaranteed delivery, Preventing Data Loss

H

handshakes, Leading Teams by Example
happy path, Core System
Hawthorne effect, Leveraging Checklists
Hexagonal architecture, Hexagonal architecture-Hexagonal architecture
hugging, Leading Teams by Example

I

implementation coupling, Architectural Quanta and Granularity
implementation, intersection of architecture with, Architecture and Implementation-Architectural Constraints
    architectural constraints, Architectural Constraints
    operational concerns, Operational Concerns-Operational Concerns
    structural integrity, Structural Integrity-Structural Integrity
implicit architectural characteristics
    importance of, Architectural Characteristics and System Design
    modularity and, Modularity
    Silicon Sandwiches (kata example), Implicit Characteristics-Implicit Characteristics
in-memory replicated cache, Operational Concerns
incoming coupling (see afferent coupling)
infrastructure services, Infrastructure services
infrastructure, intersection of architecture with, Architecture and Infrastructure-Architecture and Infrastructure
initiating event, Topology
instability (modularity metric), Core Metrics
insurance claims processing use case, Examples and Use Cases
interfaces, defined, Architectural Significance
International Organization for Standards (ISO) definitions for architectural characteristics, Cross-Cutting Architectural Characteristics-Cross-Cutting Architectural Characteristics
interpersonal skills, Possess Interpersonal Skills
intersections of architecture, Architectural Intersections-Summary
    with data topologies, Architecture and Data Topologies-Read/Write Priority
        architectural characteristics, Architectural Characteristics
        data structure, Data Structure

database topologies, Database Topology-Database Topology
read/write priority, Read/Write Priority
with engineering practices, Architecture and Engineering Practices-Architecture and Engineering Practices
with generative AI, Architecture and Generative AI-Generative AI as an Architect Assistant
Gen AI as architect assistant, Generative AI as an Architect Assistant-Generative AI as an Architect Assistant
incorporating Gen AI into architecture, Incorporating Generative AI into Architecture
with implementation, Architecture and Implementation-Architectural Constraints
architectural constraints, Architectural Constraints
operational concerns, Operational Concerns-Operational Concerns
structural integrity, Structural Integrity-Structural Integrity
with infrastructure, Architecture and Infrastructure-Architecture and Infrastructure
with systems integration, Architecture and Systems Integration
with team topologies, Architecture and Team Topologies
with the business environment, Architecture and the Business Environment-Architecture and the Business Environment
with the enterprise, Architecture and the Enterprise
Inverse Conway Maneuver, Architecture Partitioning
Irrational Artifact Attachment antipattern, Tools
ISO (International Organization for Standards) definitions for architectural characteristics, Cross-Cutting Architectural Characteristics-Cross-Cutting Architectural Characteristics

J

Jakarta Messaging API, Preventing Data Loss
Java, Three-tier
Java 1.0, Defining Modularity
JDepend, Cyclic dependencies

K

katas, Extracting Architectural Characteristics
(see also Silicon Sandwiches; Going Green)
origins, Extracting Architectural Characteristics

working with, Working with Katas
key-based event payloads, Key-based event payload-Key-based event payload
knowledge pyramid, Technical Breadth-Technical Breadth
known unknowns, Architecture and the Business Environment

L

Lack of Cohesion in Methods (LCOM), Cohesion-Cohesion
large language models (LLMs), Architecture and Generative AI-Generative AI as an Architect Assistant
last participant support (LPS), Preventing Data Loss
last responsible moment, The Covering Your Assets Antipattern
Law of Demeter (Principle of Least Knowledge), The Law of Demeter-The Law of Demeter
Law of Diminishing Returns, Leveraging Checklists
laws of software architecture, The Laws of Software Architecture, Revisited-The Spectrum Between Extremes
defining, Laws of Software Architecture-Laws of Software Architecture
First Law: everything in software architecture is a trade-off, Laws of Software Architecture, First Law: Everything in Software Architecture Is a Trade-Off-Second Corollary: You Can't Do It Just Once
example: shared library versus shared service, Shared Library Versus Shared Service-Shared Library Versus Shared Service
example: synchronous versus asynchronous messaging, Synchronous Versus Asynchronous Messaging-Synchronous Versus Asynchronous Messaging
first corollary: missing trade-offs, First Corollary: Missing Trade-Offs-First Corollary: Missing Trade-Offs
second corollary: trade-off analysis as primary job for architects, Second Corollary: You Can't Do It Just Once
Second Law: why is more important than how, Laws of Software Architecture, Second Law: Why Is More Important Than How
Out of Context antipattern, Out of Context Antipattern
Third Law: decisions exist on a spectrum, Laws of Software Architecture, The Spectrum Between Extremes
layered architectural style, Layered Architecture Style-Examples and Use Cases

cloud considerations, Cloud Considerations
common risks, Common Risks
data topologies, Data Topologies
examples and use cases, Examples and Use Cases
governance, Governance
self-assessment questions, Chapter 10: Layered Architecture Style
style characteristics, Style Characteristics-When Not to Use
style specifics, Style Specifics-Adding Layers
adding layers, Adding Layers-Adding Layers
layers of isolation, Layers of Isolation-Layers of Isolation
team topology considerations, Team Topology Considerations
topology, Topology-Topology
when not to use, When Not to Use
when to use, When to Use
layered stack, Providing Guidance
layers of isolation, Layers of Isolation
layers, using semantically when diagramming, Tools
LCOM (Lack of Cohesion in Methods), Cohesion-Cohesion
leadership, The Software Architect as a Leader-Leading Teams by Example
(see also team topologies)
4 Cs of architecture and, The 4 Cs of Architecture-The 4 Cs of Architecture
as requirement for software architect, Possess Interpersonal Skills, The Software Architect as a Leader-Leading Teams by Example
balancing pragmatism and vision, Be Pragmatic, Yet Visionary-Be Pragmatic, Yet Visionary
integrating with the development team, Integrating with the Development Team-Integrating with the Development Team
leading teams by example, Leading Teams by Example-Leading Teams by Example
leadership skills
self-assessment questions, Chapter 25: Negotiation and Leadership Skills
leaf nodes, Defining Logical Components
least worst architecture, Trade-Offs and Least Worst Architecture-Trade-Offs and Least Worst Architecture
LLMs (large language models), Leveraging Generative AI and LLMs in Architectural Decisions, Architecture and Generative AI-Generative AI

as an Architect Assistant
logical architecture, creating
analyzing architectural characteristics, Analyzing Architectural Characteristics
analyzing roles/responsibilities, Analyzing Roles and Responsibilities-Analyzing Roles and Responsibilities
assigning user stories to components, Assigning User Stories to Components-Assigning User Stories to Components
creating, Creating a Logical Architecture-Restructuring Components
identifying core components
Actor/Action approach, The Actor/Action approach-The Actor/Action approach
Entity Trap antipattern, The Entity Trap-The Entity Trap
Workflow approach, The Workflow approach
physical architecture versus, Logical Versus Physical Architecture-Logical Versus Physical Architecture
restructuring components, Restructuring Components
logical components, Defining Software Architecture
(see also component-based thinking)
defining, Defining Logical Components-Defining Logical Components
restructuring, Restructuring Components
LPS (last participant support), Preventing Data Loss

M

maintainability, ISO definition, Cross-Cutting Architectural Characteristics
mean time to recover (MTTR), Common Risks, Style Characteristics
mediated event-driven architecture, Mediated Event-Driven Architecture-Mediated Event-Driven Architecture
medical monitoring system use case, Examples and Use Cases-Examples and Use Cases
meetings, when integrating with development team, Integrating with the Development Team-Integrating with the Development Team
message bus, Orchestration engine and message bus
microfrontends, Frontends
microkernel architectural style, Microkernel Architecture Style-Examples and Use Cases

architecture characteristics ratings, Architecture Characteristics Ratings-Architecture Characteristics Ratings
cloud considerations, Cloud Considerations
common risks, Common Risks
contracts, Contracts
core system, Core System-Core System
data topologies, Data Topologies
examples and use cases, Examples and Use Cases-Examples and Use Cases
plug-in components, Plug-In Components-Plug-In Components
registry, Registry
self-assessment questions, Chapter 13: Microkernel Architecture Style
Silicon Sandwiches case study, Microkernel-Microkernel
spectrum of microkern-ality, The Spectrum of "Microkern-ality"
team topology considerations, Team Topology Considerations
topology, Topology
microservices architectural style, Styles Versus Patterns, Microservices Architecture-Examples and Use Cases
cloud considerations, Cloud Considerations
common risks, Common Risks-Common Risks
data topologies, Data Topologies-Data Topologies
enforced heterogeneity, Communication
examples and use cases, Examples and Use Cases-Examples and Use Cases
governance, Governance-Governance
intersection of architecture and infrastructure, Architecture and Infrastructure
self-assessment questions, Chapter 18: Microservices Architecture
style characteristics, Style Characteristics-Style Characteristics
style specifics, Style Specifics-Transactions and Sagas
API layer, API Layer
bounded context, Bounded Context
choreography and orchestration, Choreography and Orchestration-Choreography and Orchestration
communication, Communication-Communication
data isolation, Data Isolation
frontends, Frontends-Frontends
granularity, Granularity

operational reuse, Operational Reuse-Operational Reuse
transactions and sagas, Transactions and Sagas-Transactions and Sagas
team topology considerations, Team Topology Considerations-Team Topology Considerations
topology, Topology-Topology
modular monolith architectural style, The Modular Monolith Architecture Style-Examples and Use Cases
cloud considerations, Cloud Considerations
common risks, Common Risks
data topologies, Data Topologies
domain partitioning in, Architecture Partitioning
examples and use cases, Examples and Use Cases-Examples and Use Cases
governance, Governance-Governance
Silicon Sandwiches case study, Modular Monolith-Modular Monolith
style characteristics, Style Characteristics-When Not to Use
when not to use, When Not to Use
when to use, When to Use
style specifics, Style Specifics-Mediator approach
modular structure, Modular Structure
module communication, Module Communication-Mediator approach
monolithic structure, Monolithic Structure
team topology considerations, Team Topology Considerations
topology, Topology
modular programming languages, Defining Modularity
modularity, Modularity-From Modules to Components
defining, Defining Modularity-Defining Modularity
granularity versus, Modularity Versus Granularity
Java package namespace, Defining Modularity
metrics, Measuring Modularity-Connascence properties
abstractness, Core Metrics
cohesion, Cohesion-Cohesion
connascence, Connascence-Connascence properties
coupling, Coupling
Distance from the Main Sequence, Distance from the Main Sequence-Distance from the Main Sequence

instability, Core Metrics
limitations of, Distance from the Main Sequence
modular reuse before classes, Defining Modularity
modules to components, From Modules to Components
self-assessment questions, Chapter 3: Modularity
module communication, Module Communication-Mediator approach
mediator approach, Mediator approach
peer-to-peer, Peer-to-peer approach
monolithic architectures
case study: Silicon Sandwiches, Monolith Case Study: Silicon Sandwiches-Microkernel
microkernel architecture, Microkernel-Microkernel
modular monolith, Modular Monolith-Modular Monolith
distributed architectures versus, Monolithic Versus Distributed Architectures-Fallacy #11. Observability is optional (for distributed architectures)
scoping and, Scoping and Architectural Style
monolithic database topology, Monolithic Database Topology-Monolithic Database Topology
monolithic frontend, Frontends
MTTR (mean time to recover), Common Risks, Style Characteristics

N

n-tiered architectural style (see layered architectural style)
namespaces, Defining Modularity
near-cache model, Near-cache considerations
negotiation skills, Negotiation and Facilitation-Negotiating with Developers
negotiating with business stakeholders, Negotiating with Business Stakeholders-Negotiating with Business Stakeholders
negotiating with developers, Negotiating with Developers-Negotiating with Developers
negotiating with other architects, Negotiating with Other Architects-Negotiating with Other Architects
self-assessment questions, Chapter 25: Negotiation and Leadership Skills
Netflix, Chaos Monkey, Distance from the Main Sequence fitness function
non-functional characteristics, Architectural Significance

non-functional requirements, longevity of term, Architectural Characteristics Defined
nondeterministic workflows, Style Characteristics

O

object-oriented programming languages, Defining Modularity
observability, Fallacy #11. Observability is optional (for distributed architectures)
online auction system use case, Online Auction System
(see also Going, Going, Gone (GGG))
open layers, Layers of Isolation
operational architectural characteristics, Operational Architectural Characteristics
operational concerns, Operational Concerns-Operational Concerns
operational coupling, separating domain coupling from, Separating Domain and Operational Coupling-Service Mesh
operational measures, of architectural characteristics, Operational Measures-Operational Measures
orchestration
choreography versus, Orchestration Versus Choreography-Orchestration Versus Choreography
in microservices, Choreography and Orchestration-Choreography and Orchestration
orchestration engine, Orchestration engine and message bus
orchestration-driven service-oriented architecture, Orchestration-Driven Service-Oriented Architecture-Examples and Use Cases
cloud considerations, Cloud Considerations
common risks, Common Risks
data topologies, Data Topologies
declarative transactions, Data Topologies
examples and use cases, Examples and Use Cases-Examples and Use Cases
governance, Governance-Governance
self-assessment questions, Chapter 17: Orchestration-Driven Service-Oriented Architecture
style characteristics, Style Characteristics-Style Characteristics
style specifics, Style Specifics-Reuse...and Coupling
application services, Application services
business services, Business services

enterprise services, Enterprise services
infrastructure services, Infrastructure services
message flow, Message flow
orchestration engine and message bus, Orchestration engine and message bus
reuse and coupling, Reuse…and Coupling-Reuse…and Coupling
taxonomy, Taxonomy-Message flow
team topology considerations, Team Topology Considerations
topology, Topology
orthogonal coupling, Service Mesh
outgoing coupling (see efferent coupling)

P

package namespace (Java 1.0), Defining Modularity
packaging, modularity and, Defining Modularity
patient monitoring system use case, Examples and Use Cases-Examples and Use Cases
patterns in architecture, architectural styles versus, Styles Versus Patterns-Styles Versus Patterns
performance efficiency, ISO definition, Cross-Cutting Architectural Characteristics
performance, multiple definitions of, Operational Measures
persisted queues, Data Synchronization and Consistency
personal boundaries, Leading Teams by Example
Pets.com, Architecture and Infrastructure
physical architecture
defined, Logical Versus Physical Architecture
logical architecture versus, Logical Versus Physical Architecture-Logical Versus Physical Architecture
pipeline architectural style, Pipeline Architecture Style-Examples and Use Cases
cloud considerations, Cloud Considerations-Cloud Considerations
common risks, Common Risks
data topologies, Data Topologies
examples and use cases, Examples and Use Cases-Examples and Use Cases
filters, Filters
governance, Governance-Governance
pipes, Pipes

self-assessment questions, Chapter 12: Pipeline Architecture Style
style characteristics, Style Characteristics-When Not to Use
when not to use, When Not to Use
when to use, When to Use
team topology considerations, Team Topology Considerations
topology, Topology
platform teams, defined, Team Topologies and Architecture
plug-in components
dependencies, Plug-In Dependencies
microkernel architectural style, Plug-In Components-Plug-In Components
plug-in registry, Registry
pluralistic ignorance, Pluralistic Ignorance
POCs (proofs-of-concept), Balancing Architecture and Hands-On Coding
point-to-point communication, Plug-In Components
poison event, Topology
politics, understanding/navigating, Understand and Navigate Politics
portability, ISO definition, Cross-Cutting Architectural Characteristics
pragmatism, defined, Be Pragmatic, Yet Visionary
Principle of Least Knowledge (Law of Demeter), The Law of Demeter-The Law of Demeter
process loss, Process Loss
process, defined, Architecture and Engineering Practices
producer filters, Filters
product-based applications, microkernel architecture for, Microkernel Architecture Style
productivity flow, Integrating with the Development Team
proofs-of-concept (POCs), Balancing Architecture and Hands-On Coding
protocol-aware heterogeneous interoperability, Communication
pseudosynchronous communications (request-reply messaging), Request-Reply Processing-Request-Reply Processing

R

read/write priority, Read/Write Priority
reliability, ISO definition, Cross-Cutting Architectural Characteristics
replicated caching, Replicated and distributed caching
replication latency (RL), Data Collisions

representational consistency, Diagramming Architecture
Request for Comments (RFC), Status
request-based application, Event-Driven Architecture Style
request-reply messaging, Request-Reply Processing-Request-Reply Processing
residuality theory, Architecture and the Business Environment
risk (see architecture risk)
risk assessments, Risk Assessments-Risk Assessments
risk storming, Risk Storming-Phase 3: Risk Mitigation
  phase 1: identification, Phase 1: Identification
  phase 2: consensus, Phase 2: Consensus-Phase 2: Consensus
  phase 3: mitigation, Phase 3: Risk Mitigation
  use case, Risk-Storming Use Case-Security
    availability risk, Availability-Availability
    elasticity risk, Elasticity
    security risk, Security-Security
  user-story risk analysis, User-Story Risk Analysis
risk-assessment matrix, Risk Matrix-Risk Matrix
RL (replication latency), Data Collisions
rules engine, Examples and Use Cases
runtime plug-in components, Plug-In Components

S

Saga pattern, Transactions and Sagas
scalability, Explicit Characteristics
scoping (see architectural characteristics, scope of)
security
  as implicit characteristic, Implicit Characteristics
  ISO definition, Cross-Cutting Architectural Characteristics
self-assessment questions, Chapter 1: Introduction
semantic coupling, Architectural Quanta and Granularity
semantic decoupling, Broadcast Capabilities
separation of concerns, Topology
separation of technical concerns, Architecture Partitioning
serialization, three-tier architecture and, Three-tier
serverless architectural style, Cloud Considerations
service discovery, Operational Reuse
Service Mesh pattern
  microservices and, Operational Reuse

separating domain coupling and operational coupling, Service Mesh-Service Mesh
service plane, Operational Reuse
service-based architectural style, Service-Based Architecture Style-Examples and Use Cases
cloud considerations, Cloud Considerations
common risks, Common Risks
data topologies, Data Topologies-Data Topologies
examples and use cases, Examples and Use Cases-Examples and Use Cases
governance, Governance
self-assessment questions, Chapter 14: Service-Based Architecture Style
style characteristics, Style Characteristics-Style Characteristics
style specifics, Style Specifics-API Gateway Options
API Gateway options, API Gateway Options
service design and granularity, Service Design and Granularity
user interface options, User Interface Options-User Interface Options
team topology considerations, Team Topology Considerations
topology, Topology-Topology
service-level agreements (SLAs), Availability
service-level objectives (SLOs), Availability
service-oriented architecture (SOA), Style Specifics
(see also orchestration-driven service-oriented architecture)
services, origins and evolution of term, Style Specifics
Sidecar pattern, Operational Reuse-Operational Reuse, Service Mesh
Silicon Sandwiches (kata example), Kata: Silicon Sandwiches-Implicit Characteristics
explicit characteristics, Explicit Characteristics-Explicit Characteristics
implicit characteristics, Implicit Characteristics-Implicit Characteristics
monolithic architecture case study, Monolith Case Study: Silicon Sandwiches-Microkernel
microkernel architecture, Microkernel-Microkernel
modular monolith, Modular Monolith-Modular Monolith
partitioning, Kata: Silicon Sandwiches—Partitioning-Technical partitioning

domain partitioning, Domain partitioning
technical partitioning, Technical partitioning
single monolithic database topology, Monolithic Database Topology-Monolithic Database Topology
Single-Broker pattern, Broker-Domain Pattern
SLAs (service-level agreements), Availability
SLOs (service-level objectives), Availability
SOA (service-oriented architecture), Style Specifics
(see also orchestration-driven service-oriented architecture)
soft skills
architectural decisions (see architectural decisions)
diagramming architecture (see diagramming architecture)
leadership (see leadership)
making teams effective (see development teams, making effective)
negotiation (see negotiation skills)
software architects
collaboration with development team, Collaboration-Collaboration
integrating with the development team, Integrating with the Development Team-Integrating with the Development Team
leadership by (see leadership)
negotiations between, Negotiating with Other Architects-Negotiating with Other Architects
personality types, Architect Personalities-The Effective Architect
armchair architect, The Armchair Architect-The Armchair Architect
control-freak architect, The Control-Freak Architect
effective architect, The Effective Architect
role in making teams effective (see development teams, making effective)
role/expectations of, Expectations of an Architect-Understand and Navigate Politics
continually analyzing architecture, Continually Analyze the Architecture
ensuring compliance with decisions/design principles, Ensure Compliance with Decisions
interpersonal skills, Possess Interpersonal Skills
keeping current with latest technology/trends, Keep Current with Latest Trends
knowing the business domain, Know the Business Domain

making architecture decisions, Make Architecture Decisions
political skills, Understand and Navigate Politics
understanding of diverse technologies, Understand Diverse Technologies
technical breadth as requirement for, Technical Breadth-Rings
technical depth versus technical breadth, Technical Breadth
tips and techniques for deepening technical abilities, Balancing Architecture and Hands-On Coding-Balancing Architecture and Hands-On Coding
software architecture (basics)
defining, Defining Software Architecture-Defining Software Architecture
four dimensions of, Defining Software Architecture-Defining Software Architecture
laws of software architecture, Laws of Software Architecture-Laws of Software Architecture
role/expectations of software architect, Expectations of an Architect-Understand and Navigate Politics
self-assessment questions, Chapter 1: Introduction
software developers (see developers; development teams, making effective)
software release checklist, Software-Release Checklist
solutions, architecture patterns versus, Architectural Patterns
source code (see implementation)
space-based architectural style, Space-Based Architecture Style-Online Auction System
cloud considerations, Cloud Considerations
common risks, Common Risks-Data Collisions
data collisions, Data Collisions-Data Collisions
data synchronization and consistency, Data Synchronization and Consistency
frequent reads from database, Frequent Reads from the Database
high data volumes, High Data Volumes
data grid, Data Grid-Near-cache considerations
near-cache considerations, Near-cache considerations
replicated and distributed caching, Replicated and distributed caching-Replicated and distributed caching
data pumps, Data Pumps-Data Pumps

data readers, Data Readers-Data Readers
data topologies, Data Topologies
data writers, Data Writers-Data Writers
deployment manager, Deployment Manager
examples and use cases, Examples and Use Cases-Online Auction System
concert ticketing system, Concert Ticketing System
online auction system, Online Auction System
governance, Governance-Governance
messaging grid, Messaging Grid
processing grid, Processing Grid
processing unit, Processing Unit
self-assessment questions, Chapter 16: Space-Based Architecture Style
style characteristics, Style Characteristics-Style Characteristics
team topology considerations, Team Topology Considerations-Team Topology Considerations
topology, Topology-Topology
virtualized middleware, Virtualized Middleware
stability, as implicit characteristic, Implicit Characteristics
stakeholders, negotiating with, Negotiating with Business Stakeholders-Negotiating with Business Stakeholders
stale expertise, Technical Breadth
stamp coupling, Fallacy #3: Bandwidth Is Infinite, Data-based event payloads-Data-based event payloads, Governance
static connascence, Static connascence-Static connascence
static coupling, Architectural Quanta and Granularity, Static Coupling, Governance
strategic decisions, tactical decisions versus, Strategic Versus Tactical Decisions
stream-aligned teams, defined, Team Topologies and Architecture
structural architectural characteristics, Structural Architectural Characteristics
structural integrity, Structural Integrity-Structural Integrity
structured programming languages, Defining Modularity
Swarm of Gnats antipattern, The Swarm of Gnats Antipattern-The Swarm of Gnats Antipattern
synchronous communication, Synchronous Communication
as factor in choosing architectural style, Decision Criteria

in microservices, Communication
synchronous send, Preventing Data Loss
system design, architectural characteristics and, Architectural Characteristics and System Design-Architectural Characteristics and System Design
systems integration, intersection of architecture with, Architecture and Systems Integration

T

TAB (Technology Advisory Board), The Thoughtworks Technology Radar
tactical decisions, strategic decisions versus, Strategic Versus Tactical Decisions
tags, in pipeline architecture, Governance-Governance
tax preparation software use case
microkernel architecture and, Examples and Use Cases
registry for, Registry
team topologies, Team Topologies and Architecture
EDA style and, Team Topology Considerations-Team Topology Considerations
intersection of architecture with, Architecture and Team Topologies
Inverse Conway Maneuver and, Architecture Partitioning
layered architectural style and, Team Topology Considerations
microservices architectural style and, Team Topology Considerations-Team Topology Considerations
modular monolith architectural style and, Team Topology Considerations
orchestration-driven service-oriented architectural style and, Team Topology Considerations
pipeline architectural style and, Team Topology Considerations
service-based architectural style and, Team Topology Considerations
space-based architectural style and, Team Topology Considerations-Team Topology Considerations
team types that intersect with software architecture, Team Topologies and Architecture
teams, making effective (see development teams, making effective)

teamwork, as requirement for software architect, Possess Interpersonal Skills
technical breadth, Technical Breadth-Rings
20-minute rule, The 20-Minute Rule-The 20-Minute Rule
developing a personal radar for technological change, Developing a Personal Radar-Rings
technical depth versus, Understand Diverse Technologies, Technical Breadth
Thoughtworks Technology Radar, The Thoughtworks Technology Radar-Rings
technical debt, Balancing Architecture and Hands-On Coding
technical depth, technical breadth versus, Understand Diverse Technologies, Technical Breadth
technical leaders, architects as, Architecture and Engineering Practices
technical partitioning, Architecture Partitioning, Technical partitioning
microkernel architecture and, Architecture Characteristics Ratings
teams and, Architecture and Team Topologies
technical top-level partitioning, Architecture Partitioning
technically partitioned architecture
layered architecture as, Topology
pipeline architecture style as, Style Characteristics
Technology Advisory Board (TAB), The Thoughtworks Technology Radar
technology bubbles, Developing a Personal Radar
technology radar (see Thoughtworks Technology Radar)
temporal coupling, Temporal Coupling
temporary queue, Request-Reply Processing-Request-Reply Processing
tester filters, Filters
Thoughtworks Haiven, Generative AI as an Architect Assistant
Thoughtworks Technology Radar, The Thoughtworks Technology Radar-Rings
four quadrants, Parts
four rings, Rings
three-tier architecture, Three-tier
time to market, agility and, Architecture and Engineering Practices
top-level partitioning, Architecture Partitioning-Architecture Partitioning
trade-off analysis, Implicit Characteristics

trade-offs
analyzing, as part of architectural thinking, Analyzing Trade-Offs-Analyzing Trade-Offs
architecture versus design, The Significance of Trade-Offs
First Law of software architecture, First Law: Everything in Software Architecture Is a Trade-Off-Second Corollary: You Can't Do It Just Once
in architectural characteristics, Trade-Offs and Least Worst Architecture-Trade-Offs and Least Worst Architecture
transactions, in microservices, Transactions and Sagas-Transactions and Sagas
transformer filters, Filters
tuple space, Topology
20-minute rule, The 20-Minute Rule-The 20-Minute Rule

U

UML technical diagramming standard, UML
unit and functional testing checklist, Unit and Functional Testing Checklist
unitary architecture, Unitary Architecture
unknown unknowns, Architecture and the Business Environment
unstructured monolith, Common Risks
usability, ISO definition, Cross-Cutting Architectural Characteristics
user stories
assigning to logical components, Assigning User Stories to Components-Assigning User Stories to Components
user-story risk analysis, User-Story Risk Analysis

V

Vasa (Swedish warship), Limiting and Prioritizing Architectural Characteristics
visionary, defined, Be Pragmatic, Yet Visionary

W

Workflow Event pattern, Error Handling-Error Handling

Z

Zone of Pain, Distance from the Main Sequence
Zone of Uselessness, Distance from the Main Sequence

Previous chapter
Discussion Questions

Next chapter
About the Authors